MXCuBE Custom Tasks

Quite confusing, quite technical

Paweł Moćko

2026-08-14

What is the purpose

Custom tasks allow us to write custom experiments in MXCuBE that may be specific to our site, or would be tricky to share with others. They allow us to:

- avoid modifying shared code
- develop our own experiments without appeasing other facilities ;)
- they can be easily tweaked by the beamline staff if need be

How

To create a new form, there are two important pieces to implement:

- **`DATA_MODEL`** — a `Pydantic` model describing:
  - data that is supposed to come from the web app
  - what form fields will be displayed in the web app & their constraints
  - form layout
- **`QueueEntry`** — a derived class from `BaseQueueEntry`, needs 3 methods:
  - `pre_exeucute` — setup for the task
  - `execute` — actual logic to perform the task
  - `post_execute` — cleanup after the task is finished / fails.

Creating your own task

Some boilerplate – Data Model

Just a plain Pydantic model :)

```python
from pydantic import BaseModel, Field

class NewCollectionTaskParameters(BaseModel):
    num_images: int = Field(...) # ellipsis marks required fields.
    exp_time: float = Field(...)
```

Some boilerplate – Data Model

Which we can annotate:

```python
class NewCollectionTaskParameters(BaseModel):
    num_images: int = Field(
        ..., ge=0, description="number of images taken during data collection"
    )
    exposure_time: float = Field(
        default=100e-6,
        gt=0,
        lt=1,
        unit="s",
        description=(
          "Amount of time the crystal is exposed to the beam when"
          "collecting a particular image."
        ),
    )
```

Some boilerplate – Data Model

And we need to put some properties that are always required…

```
class NewCollectionDataModel(BaseModel):
    path_parameters: PathParameters
    common_parameters: CommonCollectionParamters
    collection_parameters: StandardCollectionParameters
    user_collection_parameters: NewCollectionTaskParameters
    legacy_parameters: LegacyParameters
```

Some boilerplate – Data Model

And we need to put some properties that are always required…

```
class NewCollectionDataModel(BaseModel):
    path_parameters: PathParameters
    common_parameters: CommonCollectionParamters
    collection_parameters: StandardCollectionParameters
    user_collection_parameters: NewCollectionTaskParameters
    legacy_parameters: LegacyParameters
    # and there is one required bit, to be explained later :)
    @staticmethod
    def update_dependent_fields(field_data: NewCollectionTaskParameters):
        return {}
```

Some boilerplate – Queue Model

This is quite a weird detail.

For each `QueueEntry`, there needs to be a `QueueModel` class in a 1:1 correspondence, usually it looks like this:

```
from mxcubecore.model.queue_model_objects import DataCollection

class NewCollectionQueueModel(DataCollection):
    pass
```

Some boilerplate – wiring it all together

```python
# new_collection.py
from mxcubecore.queue_entry.base_queue_entry import BaseQueueEntry

# class name has to match filename
class NewCollectionQueueEntry(BaseQueueEntry):
    NAME = "New Collection" # name in the context menu
```

Some boilerplate – wiring it all together

```
from mxcubecore.queue_entry.base_queue_entry import BaseQueueEntry,
TaskPrerequisite

class NewCollectionQueueEntry(BaseQueueEntry):
    NAME = "New Collection"
    REQUIRES = [
        TaskPrerequisite.NO_SHAPE_2D,
        TaskPrerequisite.POINT,
    ]
```

Some boilerplate – wiring it all together

```
from mxcubecore.queue_entry.base_queue_entry import BaseQueueEntry,
TaskPrerequisite

class NewCollectionQueueEntry(BaseQueueEntry):
    NAME = "New Collection"
    REQUIRES = [
        TaskPrerequisite.NO_SHAPE_2D,
        TaskPrerequisite.POINT,
    ]
    QMO = NewCollectionQueueModel
```

Some boilerplate – wiring it all together

```
from mxcubecore.queue_entry.base_queue_entry import BaseQueueEntry,
TaskPrerequisite

class NewCollectionQueueEntry(BaseQueueEntry):
    NAME = "New Collection"
    REQUIRES = [
        TaskPrerequisite.NO_SHAPE_2D,
        TaskPrerequisite.POINT,
    ]
    QMO = NewCollectionQueueModel
    DATA_MODEL = NewCollectionDataModel
```

Some boilerplate – wiring it all together

Then enable it in the `beamline.yaml` config file:

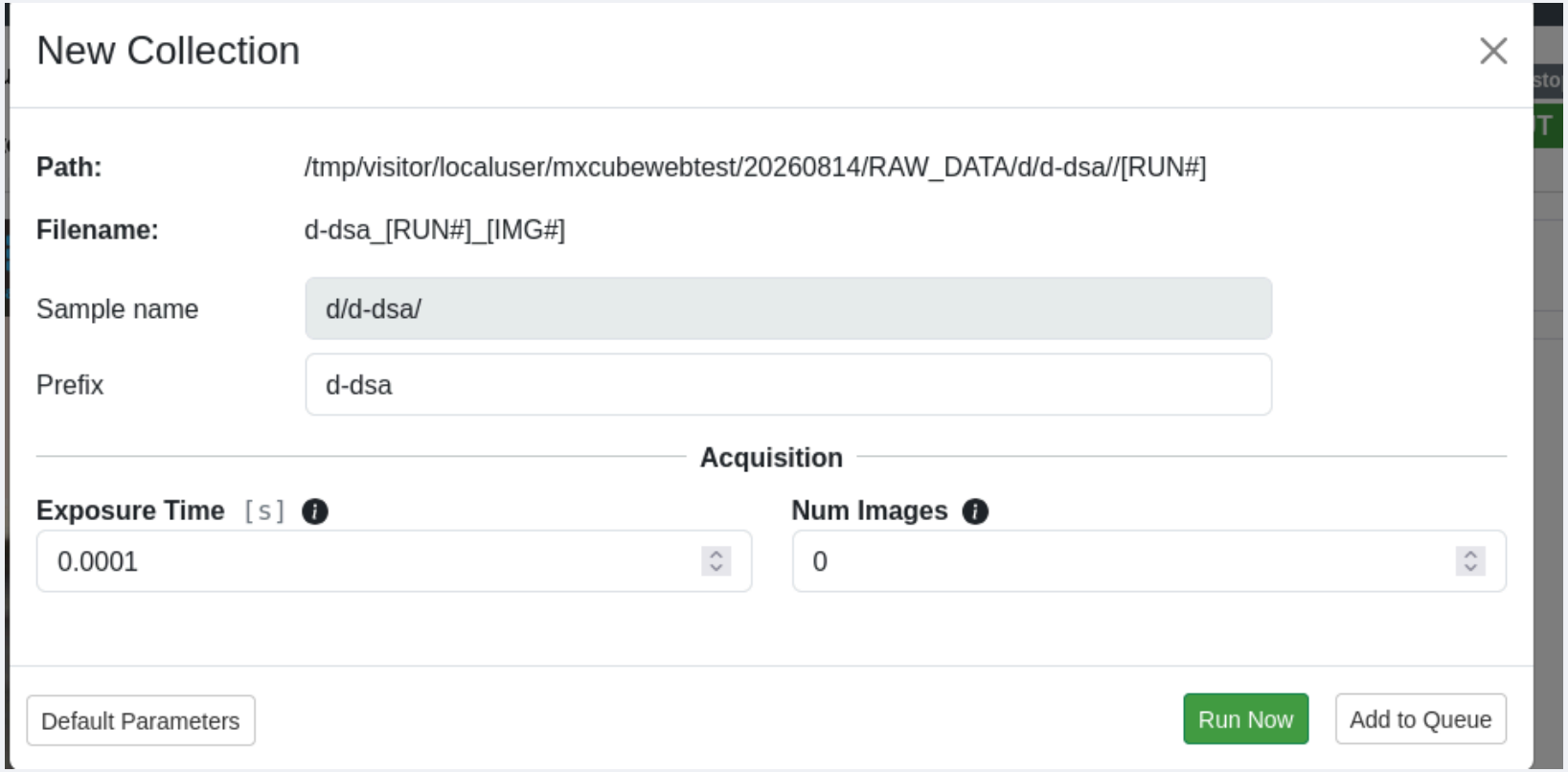
```yaml
# ...

available_methods:
    new_collection: true
    # ...
```

> the above key matches the name of the file containing `NewCollectionQueueEntry`

Some boilerplate – wiring it all together

And we should get something like this:



Defining UI layout

To change looks of a form, we can write a `ui_schema` method. the customization options include:

- order of inputs
- widgets used for inputs (e.g. dropdown menu, text input, number input, …)
- layout (width of a particular input)
- grouping inputs
- splitting groups of inputs into rows (MAX IV only code, it will definitely change “soon”)

```
class NewCollectionDataModel(BaseModel):
    # ...
    @staticmethod
    def ui_schema():
        return json.dumps({
                "ui:order": [ "num_images", "exposure_time"],
            })
```

Defining UI Layout – Example 2

```python
@staticmethod
def ui_schema():
    processing_group = {"group": "Processing"}
    col_4 = {"col": 4}
    processing_ui_options = {"ui:options": {**processing_group, **col_4}}
    return json.dumps(
        {
            "cell_a": processing_ui_options,
            "cell_b": processing_ui_options,
            "cell_c": processing_ui_options,
            "cell_alpha": processing_ui_options,
            "cell_beta": processing_ui_options,
            "cell_gamma": processing_ui_options,
        }
    )
```

Adding some behaviour

And for the task to do something, it's time to define an `execute` method.

Limitations

- On the upstream version of mxcube, the custom tasks are currently broken (thanks to some huge refactor I’ve approved :))
- There is currently no way of showing custom errors / warnings. Only generic ones like “num_images has to be > 0”.